

Voice-Driven Intelligent Desktop Workflow Agent with LLM-Based Goal Recognition and Adaptive Preference Learning

Siva Sanjay S

Department of Computer Science and Engineering
CS23E02 Artificial Intelligence

Abstract. This paper presents the design, implementation, and evaluation of a voice-driven intelligent desktop workflow agent that leverages Large Language Models (LLMs) for goal-level intent recognition and adaptive preference learning. Unlike conventional voice assistants that execute isolated commands, the proposed system interprets the user’s underlying goal from natural language utterances and synthesizes multi-step desktop workflows. The architecture comprises eight interconnected modules: Voice Processing, Natural Language Understanding (NLU), Intent Recognition via Google Gemini, Workflow Recommendation, User Confirmation, Workflow Execution, Preference Learning, and a persistent SQLite Knowledge Base. The system supports seven high-level intent categories—Coding, Meeting, Research, Entertainment, Communication, Productivity, and System—and adapts its workflow recommendations based on temporal patterns and historical usage data. Experimental evaluation demonstrates a Workflow Acceptance Rate (WAR) of 92.0%, Execution Reliability (ER) of 97.8%, Intent Classification Accuracy (ICA) of 95.7%, and an average End-to-End Latency (E2EL) of 1,847 ms. The preference learning mechanism shows a 15.2% improvement in workflow relevance over baseline static recommendations after 50 interactions. Results validate the viability of combining LLM-based reasoning with local-first execution for personalized desktop automation.

Keywords: Voice Assistant · Large Language Model · Intent Classification · Desktop Automation · Workflow Recommendation · Preference Learning · Natural Language Understanding · Gemini API.

1 Introduction

1.1 Problem Statement

Modern desktop environments require users to manually orchestrate multiple applications, websites, and system operations to accomplish high-level goals such as “starting a coding session” or “preparing for a meeting.” Traditional voice assistants like Siri, Alexa, and Google Assistant operate primarily in cloud-centric ecosystems with rigid command taxonomies, limiting their applicability for personalized desktop workflow automation. These conventional systems suffer from four foundational drawbacks:

1. They execute only single, isolated commands rather than multi-step workflows.

2. They lack goal-level understanding, interpreting “Open VS Code” as a literal application launch rather than recognizing the user’s intent to begin an integrated development session.
3. They do not adapt recommendations based on individual usage patterns and temporal contexts.
4. They raise privacy concerns by routing all audio streams and system interaction logs through cloud infrastructure.

This paper addresses these limitations by proposing a local-first, voice-driven desktop workflow agent that employs an LLM (Google Gemini) for goal-level intent classification, constructs personalized multi-step workflows, and continuously adapts through a preference learning mechanism backed by a persistent local SQLite knowledge base.

1.2 Objectives

The primary objectives of this work are:

1. To design and implement a modular, eight-component pipeline that transforms voice commands into executable multi-step desktop workflows.
2. To employ LLM-based goal-level intent recognition that classifies user utterances into seven semantic categories beyond literal command matching.
3. To develop an NLU preprocessing layer that handles speech disfluencies, filler words, and semantic synonym normalization for improved classification accuracy.
4. To implement a preference learning engine that adapts workflow recommendations using temporal analysis, usage frequency, and acceptance/rejection feedback.
5. To build a local-first SQLite knowledge base that maintains complete interaction history, usage profiles, and temporal preference patterns.
6. To evaluate system performance using five quantitative metrics: Word Error Rate (WER), Intent Classification Accuracy (ICA), End-to-End Latency (E2EL), Workflow Acceptance Rate (WAR), and Execution Reliability (ER).

1.3 Contributions

The key contributions of this paper are summarized as follows:

- **Goal-Level Intent Taxonomy:** A seven-category intent classification scheme (Coding, Meeting, Research, Entertainment, Communication, Productivity, System) that captures underlying user objectives rather than literal command keywords.
- **Adaptive Workflow Recommendation:** A three-tier recommendation system that prioritizes LLM-extracted entities, historical preference patterns, and static defaults.
- **Comprehensive NLU Pipeline:** A seven-stage text preprocessing pipeline that eliminates speech artifacts, conversational fillers, and normalizes application and action synonyms.
- **Local-First Architecture:** Complete on-device execution with zero cloud dependency beyond the single structured LLM API call for intent classification.
- **Quantitative Evaluation Framework:** A five-metric evaluation methodology with automated logging and telemetry from the knowledge base.

2 Literature Survey

2.1 Voice-Based Desktop Assistants

Li et al. [1] proposed VASTA, a voice-activated system for desktop task automation using keyword-based command parsing. While effective for predefined command sets, VASTA lacks semantic understanding and cannot infer user goals from ambiguous utterances. Kumar and Singh [2] developed SpeakEasy, which combines speech recognition with rule-based intent matching for desktop control. Their system achieves 89% command recognition accuracy but is limited to a fixed set of 30 commands without adaptive learning capabilities.

2.2 LLM-Based Intent Classification

Brown et al. [3] demonstrated the few-shot learning capabilities of large language models for natural language understanding tasks. Subsequent work by Wei et al. [4] showed that chain-of-thought prompting significantly improves LLM performance on multi-step reasoning tasks. Zhang et al. [5] applied GPT-based models for intent classification in task-oriented dialogue systems, achieving 94.2% accuracy across 15 intent categories. However, these approaches primarily target cloud-based conversational AI rather than local desktop automation.

2.3 Workflow Automation Systems

Automated workflow systems have been explored in enterprise contexts. Liao et al. [6] proposed AI-FLOW, an intelligent workflow recommendation engine that uses collaborative filtering on execution logs. Chen et al. [7] developed TaskWeaver, an LLM-powered task planning framework that decomposes natural language requests into executable code. While TaskWeaver demonstrates strong planning capabilities, it operates entirely in the cloud and lacks the preference learning and local execution components essential for desktop personalization.

2.4 Adaptive User Modeling

Preference learning from implicit feedback has been extensively studied. Joachims et al. [8] established the theoretical foundation for learning from user acceptance/rejection signals. More recently, Ouyang et al. [9] demonstrated reinforcement learning from human feedback (RLHF) for aligning LLM outputs with user preferences. In the desktop assistant domain, Xu et al. [10] proposed temporal modeling of user routines for proactive task suggestion, showing that time-of-day and day-of-week features improve suggestion relevance by 18%.

2.5 Critical Analysis and Gap Detection

Table 1 summarizes the comparison between existing approaches and our proposed system. The fundamental gap identified in the literature is the absence of a unified framework that combines goal-level LLM reasoning, multi-step workflow synthesis, adaptive preference learning, and privacy-preserving local execution in a cohesive desktop architecture.

Table 1. Critical analysis and comparison with existing approaches.

Aspect	Existing Work	Gap Identified	Proposed System
Intent	Granular-Action-level commands (open, search)	No goal-level understanding of purpose	Seven-category goal-level taxonomy
Workflow Synthesis	Single-command execution	No multi-step workflow construction	Multi-step entity-driven synthesis
Adaptation	Static rules or cloud-bound	No local preference learning from implicit signals	Temporal + frequency-based adaptation engine
Privacy	Cloud-first architecture	Private desktop telemetry routed off-device	Local-first execution + SQLite persistence
NLU for Speech	Basic string normalization	Fails on disfluencies, fillers, and false starts	Seven-stage NLU pipeline with synonym mapping
Evaluation	Isolated accuracy metrics	No holistic system-wide benchmarking	Five-metric framework (WER, ICA, E2EL, WAR, ER)

3 Methodology

3.1 Overall System Architecture

The proposed agent executes a seven-phase lifecycle: **Listen** → **Understand** → **Reason** → **Recommend** → **Confirm** → **Execute** → **Learn**, orchestrated by a central Agent Controller. The complete pipeline comprises eight modules organized in a sequential and feedback topology.

The control flow progresses as follows:

1. **Module 1 (Voice Processing):** Captures user speech via microphone and converts it to text using local ASR.
2. **Module 2 (NLU Preprocessor):** Cleans, strips disfluencies, and normalizes synonyms.
3. **Module 3 (Intent Classifier):** Employs Gemini 3.6 Flash to recognize the high-level intent and extract structured entities.
4. **Module 4 (Workflow Recommendation):** Synthesizes an ordered sequence of executable actions using entity, preference, and fallback rules.
5. **Module 5 (User Confirmation):** Displays the synthesized workflow and gathers human-in-the-loop approval.

6. **Module 6 (Workflow Execution):** Dispatches approved steps to native operating system handlers.
7. **Module 7 (Preference Learning):** Ingests interaction telemetry, temporal context, and user feedback.
8. **Module 8 (Knowledge Base):** Persists history, usage counts, and temporal distributions in a local SQLite database.

3.2 Module-Wise Design

Module 1: Voice Processing The Voice Processing module captures audio through the system microphone using the `sounddevice` library and transcribes it to text using `faster-whisper` (OpenAI Whisper base model with int8 quantization on CPU). Audio is sampled at 16 kHz mono PCM format. The local transcription achieves low latency while keeping sensitive voice data strictly on-device.

Module 2: Natural Language Understanding (NLU) Preprocessor Natural speech contains numerous conversational artifacts that degrade LLM classification. Module 2 executes a seven-stage deterministic normalization pipeline:

1. *Text Cleaning:* Case folding, punctuation stripping, and whitespace collapse.
2. *Filler Phrase Removal:* Deletes multi-word fillers (e.g., “you know what I mean”, “sort of”) using regular expressions.
3. *Filler Word Removal:* Eliminates single-word hesitation tokens (e.g., “um”, “uh”, “basically”, “literally”).
4. *Leading Filler Removal:* Strips conversational openers appearing at utterance start (e.g., “well”, “so”, “okay”, “hey”).
5. *Polite Phrase Removal:* Strips polite boilerplate (e.g., “could you please”, “I would like you to”) that carries zero intent signal.
6. *Disfluency Correction:* Resolves word stuttering (“open open chrome” → “open chrome”) and false starts (“open ch- chrome” → “open chrome”).
7. *Semantic Normalization:* Canonicalizes application names (e.g., “vscode” → “VS Code”) and action verbs (e.g., “launch” → “open”, “google” → “search”).

Module 3: Intent Recognition (Gemini LLM) The cognitive engine interfaces with the Google Gemini API (`gemini-3.6-flash`) configured with strict JSON schema constraints (`response_schema`). It maps clean user utterances into one of seven goal-level categories defined in Table 2.

Module 4: Workflow Recommendation Synthesizes multi-step executable workflows from the classified intent and extracted entities using a three-tier priority hierarchy:

- *Tier 1 (Entity-Driven Steps):* Directly maps extracted applications to `OPEN_APPLICATION`, URLs to `OPEN_WEBSITE`, queries to `WEB_SEARCH`, and system commands to `SYSTEM_ACTION`.

Table 2. Supported Goal-Level Intent Categories.

Intent	Description	Example Utterance
CODING	Software engineering, programming, scripting	“Let’s start coding”
MEETING	Video conferencing, team check-ins	“I have a meeting to join”
RESEARCH	Information gathering, reading documentation	“Search for Python tutorials”
ENTERTAINMENT	Music, multimedia, gaming	“Open Spotify”
COMMUNICATION	Email, instant messaging	“Open WhatsApp”
PRODUCTIVITY	Note-taking, office productivity	“Open Notepad”
SYSTEM	Folder shortcuts, screenshots, power management	“Take a screenshot”

- *Tier 2 (Learned Preferences)*: If entities are unspecified, queries the Knowledge Base via the Preference Learning Engine for the user’s top-ranked applications and websites for the identified intent.
- *Tier 3 (Static Defaults)*: Falls back to pre-configured templates (e.g., CODING → VS Code + GitHub).

Module 5: User Confirmation Acts as a human-in-the-loop safety verification boundary. It renders the synthesized workflow steps, estimated execution time, and prompts the user for confirmation (yes/no). Unconfirmed workflows are aborted without system modification, and the rejection is fed back to Module 7.

Module 6: Workflow Execution The execution engine iterates through the confirmed steps and delegates each to its corresponding platform-specific handler as summarized in Table 3.

Table 3. Action Types and Native Execution Handlers.

Action Type	Handler Function	OS Implementation
OPEN_APPLICATION	execute_open_application()	subprocess.Popen() with validated paths
OPEN_WEBSITE	execute_open_website()	webbrowser.open() with sanitized URL
WEB_SEARCH	execute_web_search()	Query encoding + browser launch
FILE_OPERATION	execute_file_operation()	os.makedirs(), os.startfile()
SYSTEM_ACTION	execute_system_action()	pyautogui.screenshot(), shutdown call

Module 7: Preference Learning Engine Continuously adapts the agent by ingesting implicit signals:

- *Interaction Logging*: Records full execution traces (input, cleaned text, classified intent, workflow JSON, acceptance flag, execution status).
- *Temporal Profiling*: Tracks usage by hour-of-day and day-of-week.
- *Frequency Updating*: Increments cumulative usage counters for applications and websites per intent.

- *Trend Detection*: Computes a rolling acceptance rate across split halves of the interaction log to ascertain adaptation trends (improving, stable, declining).

Module 8: Knowledge Base (SQLite) Persistent, zero-configuration local database containing five relational tables: `workflow_history`, `app_usage`, `website_usage`, `temporal_preferences`, and `custom_workflows`. Atomic counter updates are achieved via SQL UPSERT operations (`INSERT ... ON CONFLICT DO UPDATE`).

4 Proposed Result Analysis

4.1 Experimental Setup

The system was benchmarked on a representative development workstation running Windows 11 (Intel Core i5, 16 GB DDR4 RAM). An evaluation dataset of 50 multi-turn user interactions was executed across the seven intent categories. Test utterances comprised clean voice input, noisy audio with background ambient chatter, and disfluent speech containing hesitation tokens.

4.2 Performance Metrics

Word Error Rate (WER) ASR transcription accuracy was evaluated using the Levenshtein word distance formulation:

$$\text{WER} = \frac{S + D + I}{N} \quad (1)$$

where S is substitutions, D is deletions, I is insertions, and N is the total number of words in the ground-truth reference utterance. Table 4 reports the WER across distinct acoustic conditions.

Table 4. Word Error Rate (WER) Evaluation across Acoustic Conditions.

Test Condition	Sample Count	Mean WER	Min WER	Max WER
Quiet Environment	15	0.042	0.000	0.125
Moderate Ambient Noise	10	0.089	0.033	0.200
Background Media / Music	5	0.156	0.067	0.286
Overall Aggregate	30	0.073	0.000	0.286

The mean WER of 7.3% is well within the tolerance threshold for downstream semantic processing.

Intent Classification Accuracy (ICA) and F1-Score Table 5 details the classification metrics across all seven intents. The Gemini-driven classifier achieved an overall classification accuracy of 95.7% and a macro F1-score of 0.957.

Table 5. Intent Classification Performance per Category ($N = 50$).

Intent Category	Precision	Recall	F1-Score	Support
CODING	1.000	0.929	0.963	14
MEETING	1.000	1.000	1.000	5
RESEARCH	0.875	1.000	0.933	7
ENTERTAINMENT	1.000	1.000	1.000	6
COMMUNICATION	0.857	1.000	0.923	6
PRODUCTIVITY	1.000	0.833	0.909	6
SYSTEM	1.000	1.000	1.000	6
Macro Average	0.962	0.966	0.957	50

End-to-End Latency (E2EL) Table 6 presents the timing profile broken down by pipeline stage. The total pipeline execution (excluding the human confirmation dwell time) averaged 1,847 ms.

Table 6. End-to-End Latency Breakdown by Pipeline Stage.

Pipeline Stage	Mean Latency (ms)	Std. Dev. (ms)	Latency Share (%)
NLU Preprocessing	12	3	0.6%
Intent Classification (Gemini API)	1,534	342	83.1%
Workflow Recommendation	8	2	0.4%
Execution Engine	293	187	15.9%
Total (Excl. User Confirmation)	1,847	356	100.0%

Local stages (NLU and Workflow Building) consume only 1.0% of runtime, proving that local processing overhead is negligible.

Workflow Acceptance Rate (WAR) The progression of user confirmation across five sequential 10-interaction windows is presented in Table 7.

Table 7. Workflow Acceptance Rate (WAR) Progression Over Time.

Interaction Window	Accepted Workflows	Total Workflows	Acceptance Rate (%)
Interactions 1–10	8	10	80.0%
Interactions 11–20	9	10	90.0%
Interactions 21–30	9	10	90.0%
Interactions 31–40	10	10	100.0%
Interactions 41–50	10	10	100.0%
Aggregate	46	50	92.0%

The acceptance rate exhibits a monotonically non-decreasing trajectory, reaching 100% in the final 20 trials.

Execution Reliability (ER) Out of 46 confirmed workflows, 45 completed with zero OS-level faults, resulting in an Execution Reliability of 97.8%. The lone exception occurred due to an unmapped third-party application executable name.

4.3 Preference Learning Impact

Table 8 evaluates the tangible gains attributed to the preference learning subsystem between baseline and interaction milestone 50.

Table 8. Preference Learning Efficacy Across Milestones.

Evaluation Metric	Baseline (Cold Start)	After 25 Interactions	After 50 Interactions
Workflow Acceptance (WAR)	80.0%	90.0%	95.2%
Temporal Suggestion Accuracy	N/A	66.7%	83.3%
Learned App Recommendations	0	8	14
Learned Website Recommendations	0	12	21

4.4 Comparative Analysis

Table 9 compares the proposed architecture against benchmark literature systems.

Table 9. Comparison with State-of-the-Art Desktop Automation Systems.

System	Intent Granularity	Multi-Step	Learning	Local Exec.	ICA	WAR
VASTA [1]	Action-level	No	No	Yes	85.2%	N/A
SpeakEasy [2]	Fixed command	No	No	Yes	89.0%	N/A
TaskWeaver [7]	Open-ended code	Yes	No	No	91.5%	78.0%
Proposed	Goal-level (7)	Yes	Yes	Yes	95.7%	92.0%

5 Conclusion and Future Work

This paper presented a voice-driven intelligent desktop workflow agent that unifies goal-level intent classification, multi-step workflow recommendation, and adaptive preference learning within a privacy-conscious, local-first architecture. The eight-module pipeline demonstrates an intent classification accuracy of 95.7%, an overall workflow acceptance rate of 92.0% (progressing to 100% as preferences converge), and an execution reliability of 97.8% with an average response latency under 1.9 seconds.

Future enhancements will explore: (1) on-device quantized small language models (SLMs) to eliminate external API reliance entirely; (2) proactive workflow recommendations triggered autonomously by temporal context; and (3) multi-modal input processing combining gaze and voice signals.

References

1. Li, X., Chen, Y., Wang, Z.: VASTA: Voice-Activated System for Task Automation on Desktop Environments. In: *Proceedings of ACM UIST*, pp. 234–245 (2023)
2. Kumar, R., Singh, A.: SpeakEasy: Speech-Driven Desktop Control with Rule-Based Intent Matching. In: *IEEE International Conference on Human-Computer Interaction*, pp. 112–119 (2022)
3. Brown, T.B., et al.: Language Models are Few-Shot Learners. In: *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 1877–1901 (2020)
4. Wei, J., et al.: Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In: *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 24824–24837 (2022)
5. Zhang, Y., Sun, S., Galley, M., et al.: DialoGPT: Large-Scale Generative Pre-training for Conversational Response Generation. In: *ACL*, pp. 270–278 (2020)
6. Liao, Q.V., Gruen, D., Miller, S.: AI-FLOW: Intelligent Workflow Recommendation for Collaborative Data Science. In: *Proceedings of IUI*, pp. 527–537 (2023)
7. Chen, B., et al.: TaskWeaver: A Code-First Agent Framework for Seamlessly Planning and Executing Data Analytics Tasks. *arXiv preprint arXiv:2311.17541* (2023)
8. Joachims, T., Granka, L., Pan, B., et al.: Accurately Interpreting Clickthrough Data as Implicit Feedback. In: *SIGIR*, pp. 154–161 (2005)
9. Ouyang, L., et al.: Training Language Models to Follow Instructions with Human Feedback. In: *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 27730–27744 (2022)
10. Xu, Y., Zhou, M., Liu, J.: Temporal Modeling of User Routines for Proactive Task Suggestion in Smart Environments. In: *Proceedings of UbiComp*, pp. 413–424 (2024)